



Sugestões UX & Design Plataforma MB

Melhorias específicas baseadas nos models, histórias de usuário e roadmap atual do projeto. Todos os exemplos de código são compatíveis com django-unfold, sem dependências adicionais.

- django-unfold
- Fases 0–3  concluídas
- Fases 4–6 em andamento
- Sem migrations necessárias

DATA	VERSÃO	STACK	SUGESTÕES
Maio 2026	v2 — com contexto real	Django + Unfold + Neon + R2 + Railway	7 melhorias prioritizadas

Índice

☉	Contexto & status atual	Fases 0–3 concluídas, 4–6 por vir	Contexto
1	Dashboard patrimonial	DASHBOARD_CALLBACK com KPIs reais	Alta
2	Sidebar com domínio real	Ícones, grupos semânticos e badges dinâmicos	Alta
3	ContratoAdmin — @property no admin	status, data_fim e historico_precos já existem	Alta · fácil
4	TarefaAdmin — central operacional	Visibilidade das pendências geradas pelos crons	Alta · crítico
5	ImovelAdmin — properties calculadas	valor_por_m2, valor_interno, participacao_interna_pct	Média
6	DocumentoAdmin — alertas de vencimento	Planejar na Fase 4 (DOC-05)	Fase 4
7	Identidade visual & Environment badge	Tema índigo + badge Produção/Local	Baixa
☉	Roadmap de implementação	Sprint atual / próximo / pós-MVP	—

CONTEXTO ATUAL DO PROJETO

**Fases 0–3 concluídas**

Pessoas, Imóveis, Contratos e Garantias implementados com `django-simple-history`, validações de CPF/CNPJ, through model de participação e status calculado por `@property`. Base sólida.

**Fases 4–6 por vir**

Documentos (R2), Lembretes (crons), Relatórios IR. O admin ainda está no estado padrão — sem dashboard customizado, sidebar genérica e zero contexto visual operacional.



As sugestões abaixo respeitam os princípios do projeto: **baixa manutenção**, componentes nativos do Unfold, sem dependências extras. O responsável mora em Toronto — o sistema precisa falar por si mesmo, sem que ninguém precise "lembrar" de checar algo.

SUGESTÃO 01 — ALTA PRIORIDADE

Alta prioridade

A tela inicial atual lista apenas os modelos e "Ações recentes". Para alguém operando de Toronto, entrar no sistema e não ver imediatamente *o que está acontecendo* é uma falha crítica de UX.

●●● Plataforma MB – Dashboard · Maio 2026

IMÓVEIS

47

8 cidades · PB

ALUGADOS

31

66% ocupação

VALOR INTERNO

R\$ 8,4M

participação holding

TAREFAS ABERTAS

5

2 vencem em <7d

Imóveis por situação

AL

DI

RE

GT

VE

AL=Alugado · DI=Disponível · RE=Reforma · GT=Gestão terceiros · VE=Vendido

Tarefas urgentes

● Renovar contrato — Portal da Serra 604

em 3d

● Reajuste IGPM — Res. Mattos Brito

em 7d

● IPTU parcela 3 — Cabedelo

em 10d

● Seguro-fiança vencendo — tasta JP

em 45d

- KPI **Valor interno** = soma de `Imovel.valor_interno` (property já existe!) — visão da holding sem cálculo manual
- KPI **Tarefas abertas** com destaque para vencimentos em <7 dias via `Tarefa.objects.filter(concluida=False)`
- Gráfico de barras por `Imovel.Status` (AL/DI/RE/GT/VE/IN) usando `unfold/components/chart/bar.html` nativo
- Lista de tarefas urgentes no dashboard — conecta diretamente aos lembretes automáticos dos crons

PYTHON

core/views.py → dashboard_callback

```
def dashboard_callback(request, context):
    hoje = timezone.localdate()
    imoveis = Imovel.objects.all()
    context.update({
        "total_imoveis": imoveis.count(),
        "imoveis_alugados": imoveis.filter(status="AL").count(),
        "valor_interno_total": sum(i.valor_interno for i in imoveis if i.valor_interno),
        "tarefas_urgentes": Tarefa.objects.filter(
            concluida=False, vencimento__lte=hoje + timedelta(days=7)
        ).order_by("vencimento")[:5],
    })
    # Dados para gráfico por status
    status_data = imoveis.values("status").annotate(total=Count("id")).order_by("-total")
    context["chart_status"] = json.dumps({
        "labels": [s["status"] for s in status_data],
        "datasets": [{ "data": [s["total"] for s in status_data] }],
    })
    return context
```

SUGESTÃO 02 — ALTA PRIORIDADE

2 Sidebar com linguagem de domínio real

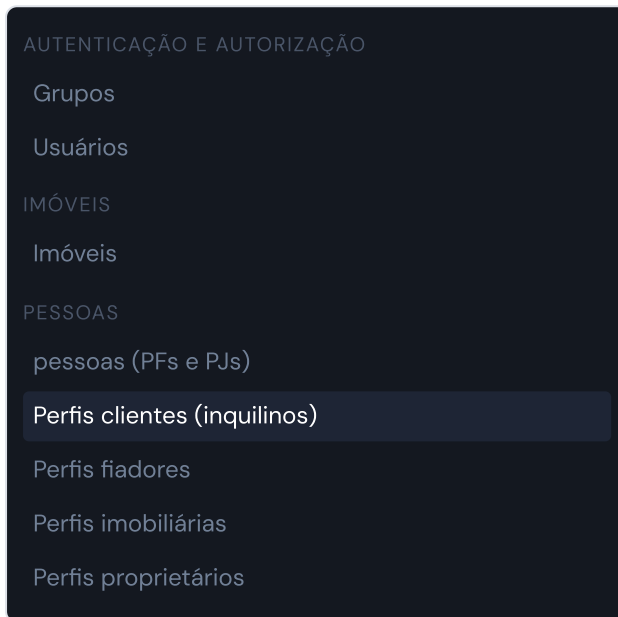
Alta prioridade

Renomear itens, adicionar ícones e reorganizar por fluxo de trabalho (não por app Django)

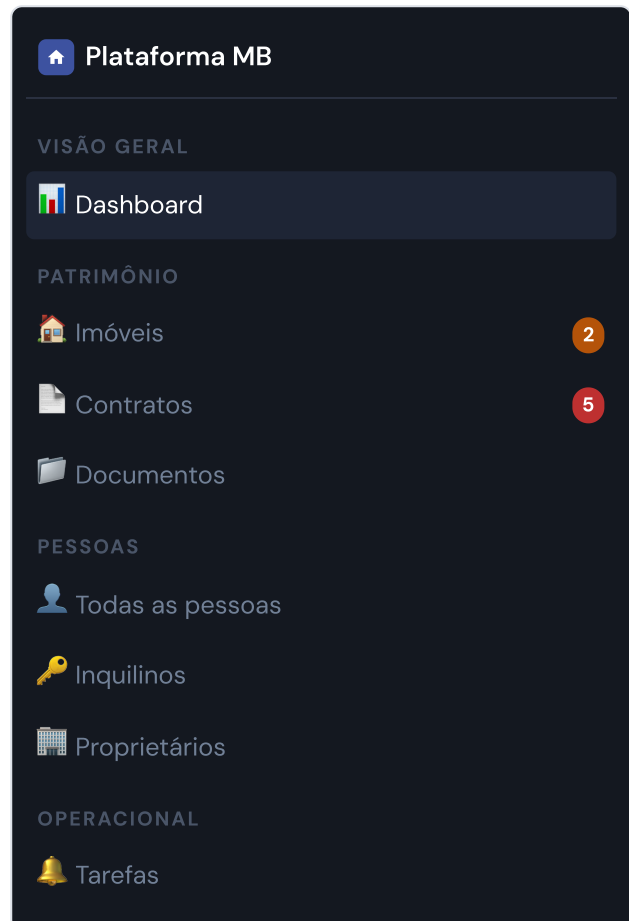


A sidebar atual espelha a estrutura de apps Django, não o fluxo mental do usuário. A reorganização é puramente de configuração em `settings.py` — nenhuma linha de Python nova além de um dicionário.

ANTES



DEPOIS



- **Contratos** com badge de vencimentos nos próximos 60d (via `badge_callback` usando `Contrato.status`)
- Separar **Tarefas** como item próprio de navegação — é o coração operacional do sistema
- Renomear `"pessoas (PFs e PJs)"` → `"Todas as pessoas"` ; perfis com nomes de papel
- Mover Autenticação para o final — não é fluxo primário de trabalho

PYTHON

settings.py — UNFOLD["SIDEBAR"]["navigation"]

```
def badge_contratos_vencendo(request):
    from datetime import date, timedelta
    alvo = date.today() + timedelta(days=60)
    count = sum(1 for c in Contrato.objects.filter(rescindido_em__isnull=True)
                if c.data_fim and c.data_fim <= alvo)
    return count or None # None oculta o badge quando zero

"SIDEBAR": {
    "show_search": True, "command_search": True,
    "navigation": [
        { "title": _("Patrimônio"), "items": [
            { "title": _("Imóveis"), "icon": "home_work", "link": reverse_lazy("admin:imoveis_imovel_changelist") },
            { "title": _("Contratos"), "icon": "description", "badge": "config.settings.badge_contratos_vencendo",
              "link": reverse_lazy("admin:contratos_contrato_changelist") },
        ]},
        { "title": _("Pessoas"), "items": [
            { "title": _("Todas as pessoas"), "icon": "group" },
            { "title": _("Inquilinos"), "icon": "vpn_key" },
            { "title": _("Proprietários"), "icon": "manage_accounts" },
        ]},
        { "title": _("Operacional"), "items": [
            { "title": _("Tarefas"), "icon": "task_alt", "link": reverse_lazy("admin:core_tarefa_changelist") },
        ]},
    ],
}
```

SUGESTÃO 03 — ALTA PRIORIDADE · FÁCIL

3 ContratoAdmin — aproveitar os @property já existentes

Alta · fácil

O model já tem `status`, `data_fim` e `historico_precos` — elas precisam aparecer no admin

MOCKUP — LISTA DE CONTRATOS ENRIQUECIDA

Contratos — 31 ativos

Contratos		Exportar IR + Novo			
IMÓVEL	INQUILINO	STATUS	VALOR	VENCIMENTO	GARANTIA
PS Portal da Serra 604	João da Silva	Ativo	R\$ 1.800	03/06/2026	Fiador
MB Res. Mattos Brito 302	Maria Oliveira	Vencendo	R\$ 2.100	15/05/2026	Seguro

- Expor `status` com `@display(label={ "Ativo": "success", "Vencendo": "warning", "Vencido": "danger" })`
- Expor `data_fim` formatada — cor vermelha automática quando <30d via `format_html`
- Tabs no form: **Dados Gerais / Financeiro / Garantia / Documentos**
- Action: **"Exportar relatório IR"** (REL-02) e **"Calcular reajuste IGPM/IPCA"** (CON-02)

PYTHON

contratos/admin.py

```
@display(description=_("Status"), label={
    "Ativo": "success", "Vencendo": "warning",
    "Vencido": "danger", "Futuro": "info",
})
def get_status_badge(self, obj):
    return obj.status # @property já implementada

@display(description=_("Vencimento"))
def get_data_fim(self, obj):
    if not obj.data_fim: return "-"
    diff = (obj.data_fim - date.today()).days
    cor = "#dc2626" if diff < 30 else ("#d97706" if diff < 60 else "inherit")
    return format_html('<span style="color:{};font-weight:500">{}</span>',
        cor, obj.data_fim.strftime("%d/%m/%Y"))
```

SUGESTÃO 04 — ALTA PRIORIDADE · CRÍTICO

4 TarefaAdmin — a central operacional do sistema

Alta · crítico

O model `Tarefa` é o coração dos crons — precisa de visibilidade de primeira classe no admin

⚠ Os crons (`verificar_contratos_vencendo`, `verificar_reajustes`, `verificar_ipitu`) criam `Tarefa` via `get_or_create`. Se o admin não mostrar as pendências com clareza, o responsável em Toronto não sabe o que está aguardando atenção.

- `list_display` : título, tipo, vencimento colorido por urgência, entidade vinculada via `GenericFK`, badge de status
- Default `get_queryset` filtra `concluida=False` — lista começa limpa, mostrando só pendências
- Filtros por `tipo` (renovacao, reajuste, iptu, seguro, irpf, manual) e `recorrencia`
- Action bulk: **"Marcar como concluídas"** — fechar lote em um clique

PYTHON

core/admin.py

```
@admin.register(Tarefa)
class TarefaAdmin(ModelAdmin):
    list_fullwidth = True
    list_display = ["titulo", "tipo", "get_vencimento", "get_entidade", "get_status_badge"]
    list_filter = ["concluida", "tipo", "recorrencia"]
    actions = ["marcar_concluidas"]

    def get_queryset(self, request):
        qs = super().get_queryset(request)
        if not request.GET.get("concluida__exact"):
            return qs.filter(concluida=False)
        return qs

    @display(description=_("Status"), label={
        "Urgente": "danger", "Pendente": "warning", "Concluída": "success",
    })
    def get_status_badge(self, obj):
        if obj.concluida: return "Concluída"
        if obj.vencimento <= date.today() + timedelta(days=7): return "Urgente"
        return "Pendente"

    @action(description=_("Marcar como concluídas"))
    def marcar_concluidas(self, request, queryset):
        queryset.update(concluida=True)
```

SUGESTÃO 05 — MÉDIA PRIORIDADE

5 ImovelAdmin — exibir o que o model já calcula

Média prioridade

Properties `valor_por_m2`, `valor_interno` e `participacao_interna_pct` existem mas não aparecem em lugar algum



PAT-03 está — as properties calculadas existem. Falta apenas expô-las no `list_display` e no form de detalhe. Sem nenhuma linha nova de lógica de negócio.

- `list_display` : nome, cidade, status (badge), área, valor `valor_por_m2` , participação interna %
- Totais no `list_after_template` : `valor_mercado` e `valor_interno` do queryset rodapé via soma filtrado
- Tabs no form: **Dados / Endereço / Proprietários (inline) / Documentos / Histórico**
- Filtros com `DropdownFilter` por cidade, estado, status e tipo

PYTHON

imoveis/admin.py

```
from unfold.contrib.filters.admin import DropdownFilter
from config.utils import formatar_moeda # já existe!

list_filter = [
    ("cidade", DropdownFilter), ("estado", DropdownFilter),
    ("status", DropdownFilter), ("tipo", DropdownFilter),
]
readonly_fields = ["valor_por_m2", "participacao_interna_pct", "valor_interno"]

@display(description=_("R$/m²"))
def get_valor_m2(self, obj):
    return formatar_moeda(obj.valor_por_m2) # @property já existente

@display(description=_("Part. interna"))
def get_participacao(self, obj):
    pct = obj.participacao_interna_pct # @property já existente
    return f"{pct:.0%}" if pct else "-"
```

SUGESTÕES 06 & 07 — FASE 4 / BAIXA PRIORIDADE

6 DocumentoAdmin — alertas de vencimento integrados

Fase 4

Planejar na Fase 4 — `Documento.vencimento` visível com badge de urgência (DOC-05)

- Badge `@display` : Vencido=danger / A vencer=warning / OK=success — mesmo padrão dos Contratos
- Inline em `ImovelAdmin` , `ContratoAdmin` e `PessoaAdmin` via `GenericTabularInline`
- Link de download via URL assinada do R2 — automático com `S3Boto3Storage`
- Dashboard: adicionar widget "Documentos a vencer em 30d" após Fase 4

7 Identidade visual & Environment badge

Baixa

Tema índigo sóbrio + badge Produção/Local + Command Palette ⌘K

Environment badge — badge vermelho
"Produção" no header evita alterações
acidentais no banco real. Uma função em
`settings.py` , detecta
`RAILWAY_ENVIRONMENT` .

Command Palette ⌘K — ativar
`"command_search": True` permite buscar
imóveis, contratos e pessoas diretamente.
Fundamental para operação remota rápida.

PYTHON

settings.py — ambiente + cores

```
def environment_callback(request):
    if os.environ.get("RAILWAY_ENVIRONMENT"):
        return ["Produção", "danger"]
    return ["Local", "info"]

"ENVIRONMENT": "config.settings.environment_callback",
"BORDER_RADIUS": "8px",
"COLORS": { "primary": {
    "500": "oklch(56.0% 0.175 250)", # índigo principal
    "950": "oklch(14.0% 0.055 250)", # sidebar fundo
}}
```

ROADMAP DE IMPLEMENTAÇÃO

AGORA — JUNTO COM FASE 4

- **S2** Sidebar com ícones e grupos reais
- **S1** DASHBOARD_CALLBACK com KPIs
- **S3** ContratoAdmin: status + data_fim
- **S4** TarefaAdmin: urgência + bulk action
- **S7** Environment badge Produção/Local

PRÓXIMO — FASE 5/6

- **S5** ImovelAdmin: totais + valor_por_m2
- **S6** DocumentoAdmin: alertas de vencimento
- **S1+** Dashboard: widget documentos
- **S3+** Action: calcular reajuste IGPM/IPCA
- **S7** Command Palette ⌘K

PÓS-MVP

- **REL** Relatório patrimonial consolidado
- **REL** Rendimentos por CPF (IRPF)
- **CON** Gerar recibo PDF (CON-06)
- **FIN** Dashboard financeiro (épico FIN)
- **S7** Logo SVG "MB" + identidade



Tudo no "Agora" pode ser feito em 1 tarde. São apenas configurações em `settings.py` e ajustes nos ModelAdmin existentes — sem novas dependências, sem novos models, sem migrations.